

Lecture 13: Breadth First Search And Depth First Search

*Instructor: Sourav Chakraborty**Scribe: Imdadul Sk, Arnab Sarkar*

1. Graph Representation

Let

$$G = (V, E)$$

be an undirected graph.

Adjacency List

For each vertex $v \in V$, define:

$$\text{Adj}[v] = \{ u \in V \mid (v, u) \in E \}$$

- Stores all neighbors of each vertex
- Space Complexity: $O(|V| + |E|)$

Adjacency Matrix

Define matrix

$$A_{ij} = \begin{cases} 1, & (i, j) \in E \\ 0, & \text{otherwise} \end{cases}$$

- Space Complexity: $O(|V|^2)$

2. BFS: Input and Output

Input

- Graph $G = (V, E)$
- Source vertex $s \in V$

Output

- All reachable vertices from source s
- Distance $d(s, v)$ for each reachable vertex

1 3. BFS Algorithm

BFS(G, s)

```
{
    for each v in V
        visited[v] = false;
    visited[s] = true;
    currentLevel = {s};
    while currentLevel is not empty
    {
        nextLevel = {};
        for each u in currentLevel
        {
            for each v in Adj[u]
            {
                if visited[v] == false
                {
                    visited[v] = true;
```

```

        add v to nextLevel;
    }
}
}
currentLevel = nextLevel;
}
}

```

Exercise 1

Let $G = (V, E)$ be a graph and $s \in V$ be a source vertex.

Modify BFS so that, for a given integer $i \geq 0$, it outputs all vertices $u \in V$ such that

$$d(s, u) = i$$

Hint: Maintain a level array and assign

$$\text{level}[v] = \text{level}[u] + 1$$

during BFS.

Lemma (Shortest Path Property)

Lemma: For every vertex u reachable from s , BFS correctly computes $d(s, u)$.

Hint:

- BFS explores vertices in non-decreasing order of distance from s
- All vertices at distance i are explored before any vertex at distance $i + 1$
- When a vertex is first discovered, no shorter path exists

- Hence, the first discovered distance is the shortest

Diameter of a Graph

Definition:

$d(u, v)$ = shortest path distance between u and v

$$\text{diam}(G) = \max_{u, v \in V} d(u, v)$$

Exercise 2

Compute the diameter of a graph $G = (V, E)$.

Time Complexity:

$$O(|V|(|V| + |E|))$$

Hint:

- Run BFS from every vertex
- Each BFS gives distances from one source
- Take the maximum over all computed distances

Exercise 3 (Tree Case)

Compute diameter when G is a tree.

Algorithm

1. Choose any vertex s
2. Run BFS from s to find a farthest vertex u
3. Run BFS from u to find the maximum distance d

4. Return d

Complexity:

$$O(|V|)$$

(two BFS traversals)

Correctness Hint:

Let (a, b) be endpoints of a diameter. Let s be any vertex and u be the farthest vertex from s .

Case 1: Paths intersect

- The path from s to u intersects the path from a to b
- Then one of a or b must be farther from s than u
- Contradiction to maximality of u

Case 2: Paths do not align

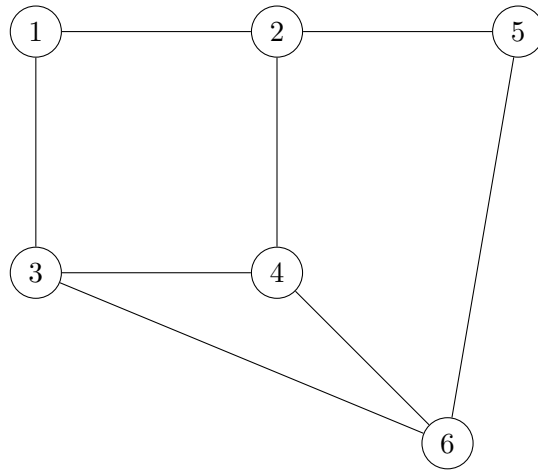
- Combining paths produces a path longer than $d(a, b)$
- Contradiction to definition of diameter

Conclusion:

- u is an endpoint of the diameter
- The second BFS finds the other endpoint

Example (Source = 1)

Original Graph:

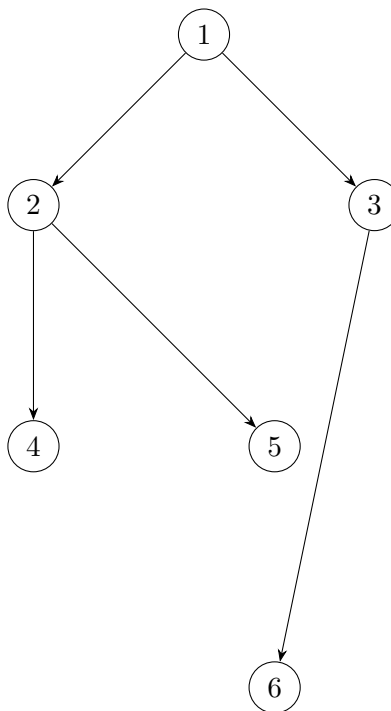


Assume adjacency order: 1: (2, 3) 2: (1, 4, 5) 3: (1, 4, 6) 4: (2, 3, 6) 5: (2, 6) 6: (3, 4, 5)

BFS Traversal from node 1:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$

BFS Tree:



Explanation:

- Level 0: 1
- Level 1: 2, 3
- Level 2: 4, 5, 6
- Node 6 is discovered from 3 (not from 4 or 5)
- Many edges (like 4-6, 5-6) are **not in BFS tree**

2 Properties of BFS Tree

1. There is **no forward edge** in a BFS tree.
2. Cross edges may exist between vertices in the **same layer** i .
3. Cross edges between layers:
 - From layer i to $i + 1$ where edges are right to left are allowed.
 - From layer i to $i + 1$ where edges are left to right are not allowed.
4. Cross edges skipping layers (e.g., $i \rightarrow i + 2$ or more) are **not allowed**.
5. Back edges are **allowed**.

Note: The number of possible BFS trees depends on:

- The ordering of vertices (adjacency list order).

3 Depth First Search (DFS)

DFS explores as deep as possible before backtracking.

4 DFS Using Stack (Iterative)

4.1 Algorithm

Initialize stack S

Push start vertex onto S

while S is not empty:

 v = top(S)

 if v has unvisited neighbor u:

 push(u)

 else:

 pop(v)

4.2 Explanation

- Stack simulates recursion.
- Push when moving deeper.
- Pop when no unvisited neighbors remain.

4.3 Discovery/Finish Times

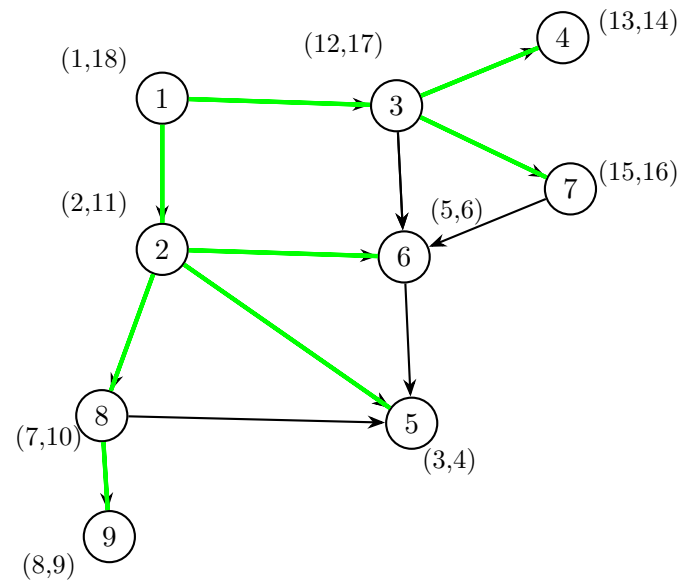
Each vertex has:

- Discovery time: $d[v]$
- Finish time: $f[v]$

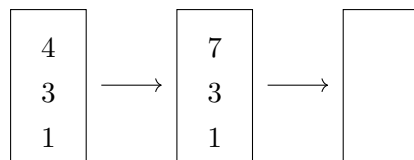
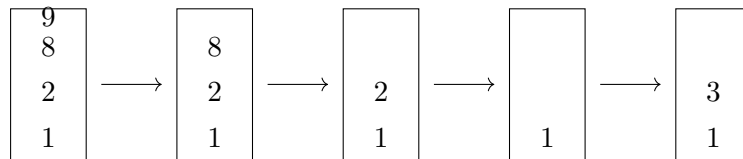
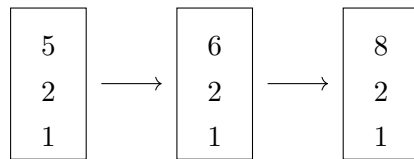
Example:

$$v_1 = (1, 18), \quad v_2 = (2, 11), \quad v_3 = (12, 17)$$

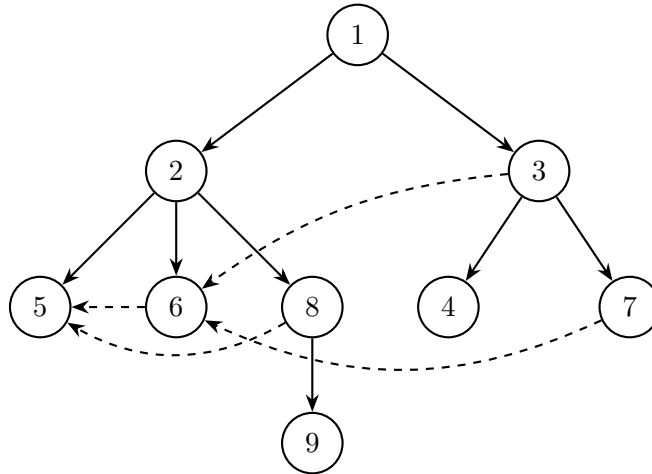
Example



Stack Evolution



DFS Tree of above example



5 Classification of Edges in DFS

When performing a Depth First Search (DFS) on a directed graph $G = (V, E)$, the search naturally partitions the edges into four distinct categories. We can classify any edge (u, v) based on the state of vertex v at the time the edge is explored, which perfectly correlates with the discovery $d[v]$ and finish $f[v]$ times.

5.1 The Four Edge Types

- 1. Tree Edges:** These are the edges in the DFS forest. An edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .

Time condition: $d[u] < d[v] < f[v] < f[u]$

- 2. Back Edges:** These edges connect a vertex u to an ancestor v in a DFS tree. Self-loops (which can occur in directed graphs) are also considered back edges. A graph contains a cycle if and only if it contains a back edge.

Time condition: $d[v] \leq d[u] < f[u] \leq f[v]$

- 3. Forward Edges:** These are non-tree edges (u, v) that connect a vertex u to a descendant v in a DFS tree.

Time condition: $d[u] < d[v] < f[v] < f[u]$ (Same as tree edges, but (u, v) is not the edge that discovered v).

- 4. Cross Edges:** All other edges. They can go between vertices in the same DFS tree (as long as one vertex is not an ancestor of the other), or they can go between vertices in entirely different DFS trees. They always point from a node explored later to a node explored earlier.

Time condition: $d[v] < f[v] < d[u] < f[u]$

Note: In an undirected graph, forward and cross edges do not exist; every edge is either a tree edge or a back edge.

5.2 Running Time:

Each edge is visited at most twice (once for each of the vertices). Each vertex is visited. So the total running time is $O(|V| + |E|)$

5.3 Note:

Because of the depth-first exploration strategy, vertex u is an ancestor of vertex v exactly in those cases where u is discovered first and v is discovered during exploring u . This is to say $d[u] < d[v] < f[v] < f[u]$.